

Callbacks in AI Agents

A LangGraph + OpenAI Tutorial

Audience: data engineers, ML engineers, and solution architects building production agentic systems.

Companion notebook: `LangGraph_Agent_Callbacks_OpenAI.ipynb` — the runnable end-to-end code for everything discussed here.

This tutorial cuts straight to what callbacks are, why they matter in production, and how to wire them into a LangGraph + OpenAI agent so you can see, measure, and trust everything the agent does. No fluff — the code you read here is the code you run.

Table of Contents

1. What Are Callbacks in Agentic Systems?
2. The Callback Event Lifecycle
3. Architecture of the Implementation
4. Wiring Callbacks In — The Two Attachment Points
5. The Custom Callback Handler (Code Walkthrough)
6. Building and Running the Graph (Code Walkthrough)
7. What a Real Run Looks Like
8. Streaming Callbacks for Real-Time UX
9. Production Patterns & Best Practices
10. Hands-On Exercises
11. Key Takeaways

1. What Are Callbacks in Agentic Systems?

Callbacks are hooks that let you intercept and observe events during an AI agent's execution lifecycle. Every time the model is invoked, a tool runs, a token is streamed, or the agent finishes, a callback method fires with a structured payload. You subclass a handler, override the methods you care about, and attach it — the framework takes care of the rest.

Why they matter in production:

- Observability — see every model prompt, tool call, and decision the agent makes, in order.
- Cost control — capture per-call token usage as the run happens, not from receipts after the fact.
- Debugging — capture the exact prompt sent to the LLM and the exact response, including tool_calls.
- Streaming UX — surface tokens to the UI as they are generated (websocket/SSE).
- Audit and compliance — timestamped event logs are a first-class artifact in regulated industries.
- Resilience — intercept LLM and tool errors early, retry or degrade gracefully instead of failing silently.

Without callbacks vs with callbacks:

Concern	Without callbacks	With callbacks
Execution visibility	Black-box: only the final answer	Full trace of every LLM and tool event
Token/cost accounting	Manual, after the fact, error-prone	Automatic per-call token usage rolled up per run
Failures	Silent — you see a wrong answer or a stack trace	Intercepted at <code>on_llm_error</code> / <code>on_tool_error</code> with context
Latency profiling	Wrap the whole call and guess	Per-node timing (start/end pairs per LLM and tool)
Streaming UX	Wait for the full response	Token-by-token via <code>on_llm_new_token</code>
Audit trail	None, or ad-hoc <code>print()</code>	Structured, timestamped, JSON-serializable event log

2. The Callback Event Lifecycle

LangChain and LangGraph share the same callback surface. Each event fires at a specific point in the run, and you override only the ones you need. The table below is the full menu.

Event	Fires when	Typical use
<code>on_llm_start</code>	Before the model is invoked	Log prompt, start latency timer
<code>on_llm_new_token</code>	Each streamed token	Live typing UX
<code>on_llm_end</code>	After the model returns	Token usage, cost, latency
<code>on_llm_error</code>	Model call fails	Alerting, retry logic

Event	Fires when	Typical use
on_tool_start	Before a tool executes	Log tool name + input
on_tool_end	After a tool returns	Capture tool output
on_tool_error	Tool raises an exception	Graceful recovery
on_agent_action	Agent picks an action	Decision tracing
on_agent_finish	Agent completes	Final output logging
on_chain_start/end	Graph/chain run boundaries	End-to-end run metrics

Execution flow in a LangGraph agent:

A user message enters the graph and is routed to the agent node, which calls the LLM — `on_llm_start` fires, then `on_llm_end` when the model returns. A conditional edge inspects the last message: if the model emitted `tool_calls`, the graph transitions to the tools node, where `on_tool_start` and `on_tool_end` fire for each tool executed. Control then loops back to the agent node for another LLM call (`on_llm_start` / `on_llm_end` again), the model now sees the tool outputs and produces a final natural-language answer. When no more `tool_calls` are emitted, the graph exits and `on_agent_finish` fires.

3. Architecture of the Implementation

The companion notebook implements a minimal but production-shaped two-node LangGraph state machine — the same ReAct-style loop that powers most real agents.

AgentState

A TypedDict with a single `messages` field annotated with the `operator.add` reducer. LangGraph uses the reducer to accumulate messages across nodes instead of overwriting state — every node that returns `{"messages": [...]}` appends to the running list.

agent node

Invokes ChatOpenAI (model="gpt-4o-mini", temperature=0) with three tools bound via `llm.bind_tools(tools)`. The model either answers directly with plain content or emits one or more `tool_calls` that the graph will execute next.

tools node

LangGraph's prebuilt ToolNode. Given the last AIMessage's `tool_calls`, it runs each requested tool in turn and appends the results as ToolMessages back into state — no custom orchestration required.

should_continue router

A plain function checked on every agent-node exit: if the last message has `tool_calls`, route to "tools"; otherwise route to END. This one function is what makes the loop terminate cleanly.

Edge from tools back to agent

`workflow.add_edge("tools", "agent")` sends control back to the LLM after tools run. This is what creates the ReAct-style think-act-observe loop.

The three tools

- `get_weather(location)` — mock weather lookup for a handful of hard-coded cities; returns a short weather string.
- `calculate(expression)` — safe-ish eval of a math expression; returns the numeric result or an error string.
- `search_database(query)` — mock knowledge base with entries for Databricks, Spark, and MLflow; returns the matching description.

4. Wiring Callbacks In — The Two Attachment Points

Callbacks attach at two levels, and in a real agent you use BOTH. Attaching only one leaves half the events unhooked.

1. Model-level attachment

Pass `callbacks=[handler]` to the `ChatOpenAI` constructor. This captures the LLM-level events — `on_llm_start`, `on_llm_new_token`, `on_llm_end`, `on_llm_error` — for every call that model makes.

```
ChatOpenAI(model="gpt-4o-mini", temperature=0, callbacks=[handler])
```

2. Run-level attachment

Pass `config={"callbacks": [handler]}` to `app.invoke(...)`. The handler is then propagated into the entire run, including the `ToolNode` — so `on_tool_start` and `on_tool_end` fire for tool executions that the model-level attachment cannot see.

```
app.invoke(state, config={"callbacks": [handler]})
```

A callback handler is a plain Python object that subclasses `BaseCallbackHandler` and overrides the methods it cares about. No metaclass tricks, no decorators — you can log, count, redact, push to Kafka, or do anything else you would do in ordinary Python.

5. The Custom Callback Handler (Code Walkthrough)

The full handler used in the companion notebook. Read it top-to-bottom first, then walk through the annotations below.

```
class AgentCallbackHandler(BaseCallbackHandler):
    """Comprehensive callback handler for agent observability."""

    def __init__(self):
        self.llm_calls = 0
        self.tool_calls = 0
        self.total_tokens = 0
        self.logs = []

    def on_llm_start(self, serialized, prompts, **kwargs):
        self.llm_calls += 1
        entry = {
```

Callbacks in AI Agents — LangGraph + OpenAI Tutorial

```
        "timestamp": datetime.now(timezone.utc).isoformat(),
        "event": "llm_start",
        "call_number": self.llm_calls,
        "prompt_preview": str(prompts)[:120],
    }
    self.logs.append(entry)
    print(f"[LLM START]   call #{self.llm_calls}")

def on_llm_end(self, response, **kwargs):
    tokens = 0
    if getattr(response, "llm_output", None):
        tokens = response.llm_output.get("token_usage", {}).get("total_tokens",
0)

    self.total_tokens += tokens
    self.logs.append({
        "timestamp": datetime.now(timezone.utc).isoformat(),
        "event": "llm_end",
        "tokens_used": tokens,
        "cumulative_tokens": self.total_tokens,
    })
    print(f"[LLM END]      tokens={tokens} | cumulative={self.total_tokens}")

def on_llm_error(self, error, **kwargs):
    self.logs.append({"event": "llm_error", "error": str(error)})

def on_tool_start(self, serialized, input_str, **kwargs):
    self.tool_calls += 1
    tool_name = serialized.get("name", "unknown")
    self.logs.append({"event": "tool_start", "tool_name": tool_name,
        "input_preview": str(input_str)[:120]})

def on_tool_end(self, output, **kwargs):
    self.logs.append({"event": "tool_end", "output_preview": str(output)
[:120]})

def on_tool_error(self, error, **kwargs):
    self.logs.append({"event": "tool_error", "error": str(error)})

def on_agent_action(self, action, **kwargs):
    self.logs.append({"event": "agent_action", "tool": action.tool})

def on_agent_finish(self, finish, **kwargs):
    self.logs.append({"event": "agent_finish"})

def get_summary(self):
    return {"total_llm_calls": self.llm_calls,
        "total_tool_calls": self.tool_calls,
        "total_tokens": self.total_tokens,
        "execution_logs": self.logs}
```

Walkthrough:

- Counters — llm_calls and tool_calls increment on the corresponding _start events, so the run's shape is visible at any moment, even mid-execution.
- total_tokens accumulator — a rolling sum across every on_llm_end. Zero out per run by instantiating a fresh handler.

- logs list — a chronologically ordered list of timestamped dicts. Every entry is plain JSON: strings, numbers, ISO timestamps. That makes it trivial to serialize to Elasticsearch, MongoDB, S3, or an observability sink.
- Token extraction — `response.llm_output["token_usage"]["total_tokens"]` is the authoritative field on the LangChain response object. Wrap it in `getattr` and `.get()` because `llm_output` is `None` for some model backends.
- Truncation (`[:120]`) — long prompts and outputs are trimmed to a short preview. Full payloads go into your log store, not into an in-memory list you print to a terminal.
- `get_summary()` — the post-run report: totals plus the full timeline. Use it in tests, CI, and dashboards.

6. Building and Running the Graph (Code Walkthrough)

Graph construction:

```
workflow = StateGraph(AgentState)
workflow.add_node("agent", agent_node)
workflow.add_node("tools", ToolNode(tools))
workflow.set_entry_point("agent")
workflow.add_conditional_edges("agent", should_continue, {"tools": "tools", "end":
END})
workflow.add_edge("tools", "agent")
app = workflow.compile()
```

Two nodes, one conditional edge, one loop-back edge, compile — that is the entire state machine. The conditional edge is the branch point that either goes to tools or exits; the tools-to-agent edge closes the ReAct loop.

Execution with run-level callback config:

```
final_state = app.invoke(
    {"messages": [HumanMessage(content=user_input)]},
    config={"callbacks": [handler]},
)
```

The config dict is where the handler enters the run. Combined with the model-level attachment on `ChatOpenAI`, this ensures both LLM events and tool events land in the same handler instance.

7. What a Real Run Looks Like

Query:

*"Get the weather for Mumbai, search the database for Spark, and calculate $25 * 4 + 10$ "*

Expected trace:

1. [LLM START] call #1 — the agent node invokes gpt-4o-mini with the user message and the three bound tools.
2. The model responds with an `AIMessage` carrying three `tool_calls` (one per requested action).
3. [TOOL START] `get_weather` — [TOOL END] returns "Hot and Humid, 32°C".
4. [TOOL START] `search_database` — [TOOL END] returns the Spark description.

5. [TOOL START] calculate — [TOOL END] returns "Result: 110".
6. [LLM START] call #2 — the agent node runs again, this time the model sees the three ToolMessages.
7. [LLM END] tokens=... | cumulative=... — the model composes and returns the final natural-language answer.
8. should_continue sees no tool_calls on the last message and routes to END; on_agent_finish fires.

A multi-tool query like this produces 2 LLM calls, 3 tool calls, and every step is timestamped in the log. Below is what the summary output looks like at the end:

```
LLM calls : 2
Tool calls: 3
Tokens    : 412
```

8. Streaming Callbacks for Real-Time UX

For chat UIs, waiting for the whole response before rendering anything is a poor experience. `on_llm_new_token` fires for each token the model produces, which you can push through a websocket or Server-Sent Events channel to render the token as it is generated.

```
class StreamingCallbackHandler(BaseCallbackHandler):
    def on_llm_new_token(self, token: str, **kwargs):
        print(token, end="", flush=True)

streaming_llm = ChatOpenAI(model="gpt-4o-mini", temperature=0,
                           streaming=True, callbacks=[StreamingCallbackHandler()])
```

Non-negotiable: `streaming=True` must be set on the model — without it, the model returns a single chunk and `on_llm_new_token` never fires. Combine the streaming handler with the observability handler by passing both in the `callbacks=` list.

9. Production Patterns & Best Practices

1. Ship logs to a real store.

The logs list is JSON-serializable by construction — push it to MongoDB, Elasticsearch, or Datadog, or send events to an observability platform like LangSmith or Langfuse. `print()` is a demo tool, not a production one.

1. Track latency.

Record `time.perf_counter()` in `on_llm_start` and compute the duration in `on_llm_end`. p50 and p95 latency per node is the first metric you need when an agent feels slow — with callbacks it is a five-line change.

1. Cost accounting.

`token_usage` also exposes `prompt_tokens` and `completion_tokens`. Multiply by the model's input and output pricing separately to get per-run and per-tenant cost. Roll up in the same store as latency.

1. Redact PII before logging.

Scrub emails, account numbers, and secrets from prompts and tool inputs before persisting logs. This is a compliance requirement in regulated industries — insurance, banking, healthcare — and doing it at the callback boundary means every downstream consumer inherits the redaction for free.

1. Async handlers for I/O.

Use `AsyncCallbackHandler` when logging to external services so callback I/O never blocks the agent loop. A blocking HTTP write inside `on_llm_end` will show up as agent latency; an async write will not.

1. Errors are data.

`on_tool_error` and `on_llm_error` are your early-warning system. Count them, tag them by tool name or model, and alert on spikes — this catches upstream API degradation and brittle tool implementations long before users complain.

1. Handler-per-run isolation.

Create a fresh handler per run (as the notebook does). Sharing a single handler across concurrent requests will interleave logs and corrupt counters — one handler per `invoke()` keeps runs cleanly attributable.

10. Hands-On Exercises

Work through these in the companion notebook. Each one is a real production pattern in miniature.

1. Latency tracking — extend the handler with per-LLM-call latency using `time.perf_counter()` in `on_llm_start` / `on_llm_end`.
2. Cost estimator — compute per-run USD cost from `prompt_tokens` and `completion_tokens` at gpt-4o-mini input/output pricing.
3. Error injection — make calculate raise on division by zero; confirm `on_tool_error` fires and the agent recovers on the next LLM turn.
4. Async logging — port the handler to `AsyncCallbackHandler` with a mocked external write and confirm the agent loop is not blocked.
5. PII redaction — regex-scrub email patterns from prompts before appending to logs.

11. Key Takeaways

Callbacks turn an agent from a black box into an observable system. LangGraph gives you the state machine, OpenAI gives you the reasoning, and the callback handler gives you the evidence of everything in between — every prompt, every tool call, every token, every error, timestamped and structured for the store of your choice.

If you build agents that will ever run in production, wire in callbacks from day one. It is a small amount of code that pays for itself the first time a run misbehaves — and it is the substrate every serious observability, cost, and compliance workflow sits on top of.

Next step: open the companion notebook

LangGraph_Agent_Callbacks_OpenAI.ipynb, set your `OPENAI_API_KEY`, and run the three example queries end-to-end.